

## Article

# A Federated Network Intrusion Detection System with Multi-Branch Network and Vertical Blocking Aggregation

Yunhui Wang <sup>1,2,†</sup>, Weichu Zheng <sup>3,†</sup>, Zifei Liu <sup>3</sup>, Jinyan Wang <sup>1,2</sup>, Hongjian Shi <sup>3,\*</sup> , Mingyu Gu <sup>4</sup> and Yicheng Di <sup>5</sup> 

<sup>1</sup> National Key Laboratory of Science and Technology on Avionics System Integration, Shanghai 200233, China; wang.yh@outlook.com (Y.W.); wangjy121@avic.com (J.W.)

<sup>2</sup> China National Aeronautical Radio Electronics Research Institute, Shanghai 200233, China

<sup>3</sup> School of Electronic Information and Electrical Engineering, Shanghai Jiao Tong University, Shanghai 200240, China; sjtu\_zwc0518@sjtu.edu.cn (W.Z.); liuzifei@sjtu.edu.cn (Z.L.)

<sup>4</sup> Sino-European School of Technology, Shanghai University, Shanghai 200444, China; 22124558@shu.edu.cn

<sup>5</sup> School of Artificial Intelligence and Computer Science, Jiangnan University, Wuxi 214122, China; diyicheng@stu.jiangnan.edu.cn

\* Correspondence: shhjwu5@sjtu.edu.cn; Tel.: +86-13717959365

† These authors contributed equally to this work.

**Abstract:** The rapid development of cloud–fog–edge computing and mobile devices has led to massive amounts of data being generated. Also, artificial intelligence technology, like machine learning and deep learning, is widely used to mine the value of the data. Specifically, detecting attacks on the cloud–fog–edge computing system using mobile devices is essential. External attacks on network press organizations led to anomaly flow in network traffic. The network intrusion detection system (NIDS) has been an effective method for detecting anomaly flow. However, the NIDS is hard to deploy in distributed networks because network flow data are kept private. Existing methods cannot obtain an accurate NIDS under such a federated scenario. To construct an NIDS while preserving data privacy, we propose a combined model that integrates binary classifiers into a whole network based on simple classifier networks to specify the type of attack on anomalous data and offer instruction to other security system components. We also introduce federated learning (FL) methods into our system and design a new aggregation algorithm named vertical blocking aggregation (FedVB) according to our model structure. Our experiments demonstrate that our system can be more effective than simple multi-classifiers in terms of accuracy and significantly reduce communication and computation overhead when applying FedVB.

**Keywords:** cloud–fog–edge computing; mobile device; artificial intelligence; network intrusion detection system; federated learning



**Citation:** Wang, Y.; Zheng, W.; Liu, Z.; Wang, J.; Shi, H.; Gu, M.; Di, Y. A Federated Network Intrusion Detection System with Multi-Branch Network and Vertical Blocking Aggregation. *Electronics* **2023**, *12*, 4049. <https://doi.org/10.3390/electronics12194049>

Academic Editor: Aryya

Gangopadhyay

Received: 25 August 2023

Revised: 9 September 2023

Accepted: 22 September 2023

Published: 27 September 2023



**Copyright:** © 2023 by the authors. Licensee MDPI, Basel, Switzerland. This article is an open access article distributed under the terms and conditions of the Creative Commons Attribution (CC BY) license (<https://creativecommons.org/licenses/by/4.0/>).

## 1. Introduction

Massive amounts of data have been collected for various applications against the background of the rapid development of cloud–fog–edge computing and mobile devices. Such data contain valuable information, but that information is hard to mine. Artificial intelligence (AI) methods, like machine learning (ML) and deep learning (DL), are widely used in this context to extract practical knowledge from these data. DL has led to significant breakthroughs in distributed networks like the Internet of Things (IoT) [1,2] and cloud computing (CC). Specifically, DL has also been used to handle privacy and security problems, like network intrusion detection, in distributed machine learning (DML) or cloud–fog–edge computing (CFEC) frameworks.

In CFEC frameworks, the increasing number of threats to network traffic can come from various sources, causing a higher likelihood of organizations being exposed to intruders. Security mechanisms, especially network intrusion detection systems (NIDSs) [3], are

essential to mitigate such issues. NIDS refers to the combination of software and hardware used to detect behaviors that endanger computer system security, such as collecting vulnerability information, causing access denial, and obtaining system control rights beyond the legal scope.

Due to static strategies and fixed behavior patterns, a traditional NIDS [4] is often limited by computation power and data distribution. The demand for more powerful and effective NIDSs is on the rise. Due to the rapid increase in network attacks and speed, we need more than traditional NIDS CFEC frameworks. There is an urgent need to reduce the computational cost of NIDSs and improve their accuracy to meet the requirements of CFEC framework security.

Specifically, deep learning (DL) has been adopted to handle this problem. It converts intrusion detection tasks into pattern recognition and classification by collecting and modeling various behaviors in a network. Specifically, NIDSs identify an attack event by monitoring raw traffic as it travels across the network, extracting useful information from it, and then classifying it by matching it to known attack characteristics or comparing it to a prototype of normal network behavior.

When configuring NIDSs in specific, more sensitive network segments, the intrusion detection system will discern suspicious threats, and the NIDS can detect attacks and unauthorized behaviors promptly. Advanced security component interaction mechanisms can also modify security policies to prevent further attacks, which enables security risks to be effectively controlled and mitigated. For increasingly complex application security threats and hybrid network attacks, the next-generation NIDS should adapt to the latest developments in attack and defense. It should accurately monitor abnormal network traffic and notify security threat managers promptly. That is, NIDSs need to have high accuracy and efficiency.

In addition, constructing an accurate NIDS requires collected network features of all kinds of network intrusions, which is difficult to satisfy in CFEC frameworks. Most of the time, mobile devices only contain the features of a few kinds of network intrusions. These features cannot be directly used to train a DL NIDS model due to privacy issues. Furthermore, CFEC frameworks introduce more concern over a device's private data. Thus, local data in any device are not visible to other devices. In such a case, obtaining an NIDS that can utilize heterogeneous private data from devices to construct an accurate DL model is urgent and essential [5]. Such a scenario is referred to as federated learning (FL) [6].

FL usually follows the server–client structure, wherein the server aggregates local models from the clients, and the clients train local models using their local dataset. Such a scenario only involves local models and prohibits raw data transmission. FL follows several steps in each communication round to train an NIDS model collaboratively:

1. The server distributes the global model to the clients to initialize their local models.
2. Each client trains the local model on their private data.
3. The clients upload their trained local models to the server.
4. The server aggregates the uploaded local models to obtain the global model for the next communication round.

There has been some research on NIDSs in the FL scenario [7]. However, those methods suffer from two problems. First, a traditional NIDS cannot accurately handle multi-class classification over network intrusion types. Most NIDSs can precisely distinguish abnormal network flows but cannot determine the type of attack. Second, previously established NIDSs require complete data samples, including data from all attacks. Such a condition is hard to satisfy, so a traditional NIDS must classify the attack type accurately.

In this paper, we propose a combined multi-branch model and apply FL methods to the problem. In the multi-branch model, we add a feature extractor and an output classifier to organize binary classification mini-models in our network. In our FL system, we design a vertical blocking aggregation algorithm (FedVB) based on the multi-branch model. The main contributions of this research work are:

1. We propose a new network intrusion detector called the multi-branch model. This model can more accurately distinguish the types of malicious traffic attacks in the network and provide instructions for other security system components.
2. We propose a new FL algorithm called FedVB. FedVB utilizes vertical blocking aggregation to reduce communication and computation volume by only transmitting effective gradient updates.
3. We evaluate the multi-branch model and FedVB on different datasets. The experimental results illustrate that our system has a higher attack type matching rate than other methods.

The remainder of the paper is as follows. Section 2 lists previous work related to our research. Section 3 describes the methodology of our proposed FL and NIDS approach. Section 4 reports the results and discussion. Section 5 concludes our research.

## 2. Related Works

With the rapid development of cloud–fog–edge computing and mobile devices, CFEC has emerged as one of the most popular technologies of the Internet today. However, the vast volume of data, the complexity of the scenarios, the reliability of communication, and the transmission security have led to the exploration of the possibility of combining CFEC with other technologies. CC [8] is a type of distributed computing that refers to the process of dividing vast amounts of data into countless smaller programs using a network ‘cloud’ and then processing and analyzing these smaller programs with a system of multiple servers to provide the results back to the user. It is a robust computing power system formed through a computer network, primarily the Internet, that can store and aggregate relevant resources and be configured on demand to provide users with personalized services [9].

### 2.1. Network Intrusion Detection

Currently, there are more sophisticated threats than ever before to network security. An increasing number of threats to network traffic can come from various sources, thus increasing the likelihood that organizations will be exposed to intruders. All the traffic on inspected segment servers is the data source for NIDSs [10]. In an Ethernet environment, the NIDS captures mixed packets on the monitored segment by setting the NIC to promiscuous mode. In general, the intrusion detection system protects the entire network segment. The positioning of the NIDS in a switched environment must be carefully designed to capture the required data. The NIDS detects possible intrusion detection behavior by matching three types of features: string features, port features, and packet header features [11].

An NIDS can detect, but it may need assistance to detect specific anomalous traffic effectively. Therefore, the NIDS algorithm must be reliable and provide high detection accuracy to minimize threats from the network. In parallel, the NIDS begins to malfunction under high load; due to its capacity limitations, it is unable to filter all incoming traffic and begins dropping packets [12]. The solution to this problem is a distributed set of multiple NIDSs that distributes the workload between them. This solution reduces the packet loss rate but allows more sophisticated attacks to pass. The demand for more powerful and effective NIDSs [13] is rising. Traditional NIDSs are no longer sufficient for CFEC frameworks due to the exponential growth of network threats and network speeds. To satisfy the needs of CFEC framework security, it is urgently necessary to lower the computing cost of NIDSs and improve their accuracy.

Software metrics can be used to assess vulnerabilities. Generating metrics using OWASP classification [14], well-defined taxonomy [15], and simulation tools [16] also helps to evaluate the overall network safety situation and provide instructions for improvement.

### 2.2. Common NIDS Usage

As mentioned above, NIDSs in CFEC frameworks need to improve their efficiency and accuracy to combat massive data traffic and computational costs. This topic has been the subject of extensive study. There are several methods for reaching these objectives. One

is to increase the size of the NIDS and, at the same time, improve parallelism so that the NIDS is large enough to cope with huge volumes of data [17]. Rouf et al. [18] propose a layered security system based on NIDSs that supports a distributed architecture to combat more sophisticated intrusions. Ryosuke et al. [19] investigate the feasibility of utilizing current security tools to produce labeled datasets that can facilitate the development of an AI-powered NIDS system. Rathore et al. [20] present a real-time intrusion detection system based on Hadoop implementation for ultra-high-speed big data environments that includes a four-tier IDS architecture. With this system, they try to detect unknown types of attacks. Lin et al. [21] propose layered parallelism to accelerate multi-string matching on several GPUs to improve the performance and throughput of multi-string matching in NIDSs. Hideya et al. [22] propose an ML-based NIDS that captures normal and malicious communications in real time and performs automatic daily updates at the target organization where the NIDS is installed.

However, as NIDSs grow in size, so do the corresponding device performance needs in a CFEC framework, and parallelism might impose more demands on communication quality. This problem is rarely mentioned in the articles above, even though their accuracy rates have mostly remained undeclined.

The other is to reduce NIDS and narrow the scenarios in which it is used. Amutha et al. [23] improve the scalability of NIDSs by removing feature selection. The suggested RNN model approach simplifies the design of NIDSs and avoids the challenging network traffic pre-processing procedure. Poltavtseva et al. [24] propose a new enterprise network association atlas (AGS) model to develop NIDSs and meet the needs of enterprise network security. Payam et al. [25] used unsupervised learning techniques applied to NIDSs to detect unknown and complex attacks in normal or encrypted communications without prior knowledge. Zayed et al. [26] propose a collaborative network intrusion detection system (C-NIDS) that detects network attacks in the cloud. It addresses new issues, such as intrusion detection in virtual networks, monitors high traffic, scalability, and resistance, and provides high accuracy.

In addition, ongoing research is being carried out to refine the training dataset for NIDSs. Lisa et al. [27] show the effects of these flaws on the benchmark datasets for NIDS, CIC-IDS-2017, and CIC-CSE-IDS-2018 and demonstrate the implications of these errors using several experiments. Shenfield et al. [28] investigate more than eleven accessible datasets since 1998. Some of these datasets need to be updated and consistent before being used. Other datasets suffer from the negative effects of low traffic variety and volumes; do not cover the range of attacks; and anonymize bundle data and payload, which cannot reflect current patterns, or they must include set and metadata. Prakash et al. [29] present an experimental method for deep and machine learning networks with hyperactive boundary improvements using a convenient IDS digital dataset (CIC-CSE-IDS-2018) considering the maximum frontier attack (PCAP). Liu et al. [30] develop an outlier detection algorithm that generates artificial outliers using the generative adversarial active learning (GAAL) framework. The discriminator learns decision limits, and the generative adversarial network's (GAN's) generator produces potential informative outliers [31].

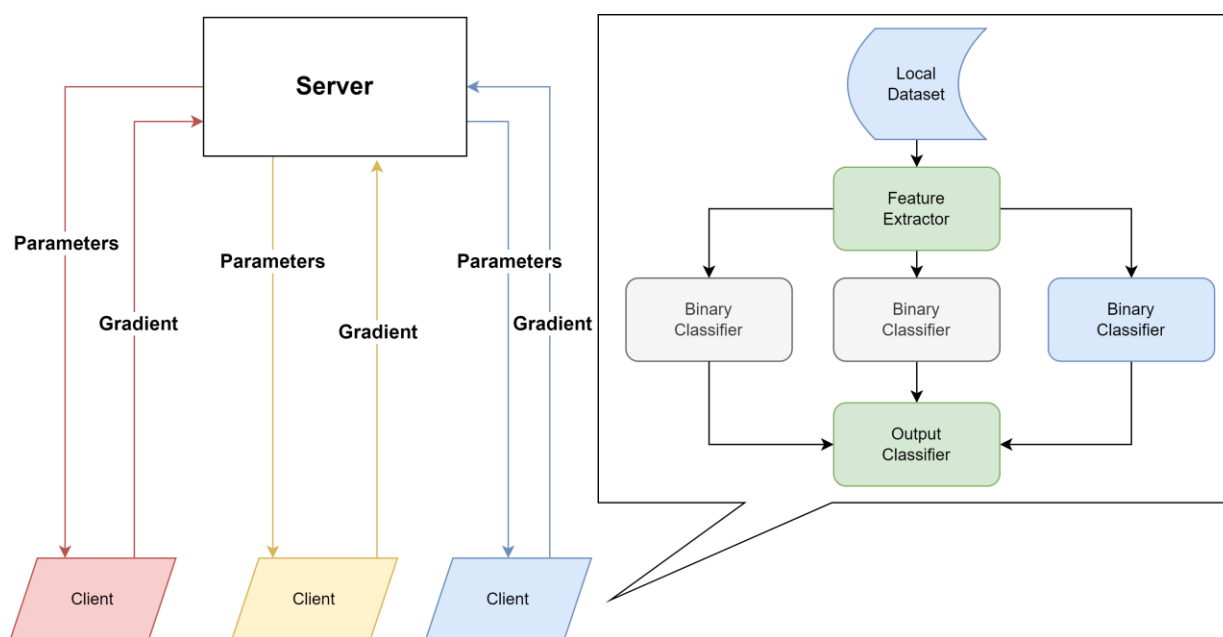
### 2.3. Federated Learning

Recently, FL [32] has received sustained attention. While adhering to the demands of user privacy protection, data security, and governmental legislation, it can successfully assist numerous firms in modeling data consumption and machine learning. FL can break down data silos and facilitate AI collaboration [33] by allowing participants to model jointly without sharing data, which makes FL an excellent DML paradigm. Many algorithms have been proposed in this area. FedAvg [34] is the most classical algorithm for joint learning of deep networks based on iterative model averaging. FedAvg introduces the federated averaging algorithm, which combines local stochastic gradient descent (SGD) on each client with a server that performs model averaging. APFL [35] reduces the dependence of generalization errors on the local data's distributional characteristics. The

model is intended to learn a personalized model that combines global and local patterns. APPLE [36] is a personalized cross-silo FL framework that adaptively learns how much each client can benefit from other models. APPLE is designed to systematically mitigate the impact of distributional variation between non-IID datasets on FL. Ditto [37] performs federated personalization by reducing reliance on global models, focusing on simultaneously improving fairness and robustness in FL. Prior to model averaging, FedBN [38] uses local batch normalization to decrease feature bias in order to reduce the impact of different data distributions on neural network training performance and speed up convergence. FedFomo [39] highlights the flexibility in personalized learning by proposing an alternative solution in which each client is federated with only relevant clients to obtain a more robust model based on client-specific goals. FedMTL [40] shows that multi-task learning is a natural choice for building personalized federated models and developing the MOCHA algorithm for multi-task learning in a federated set to address challenges related to communication, anomalous nodes, and fault tolerance. Since multi-task learning creates a model for each task, it has the disadvantage that every client must take part in every training session. For the purpose of solving collaborative networks, FedProx [41] focuses on dealing with heterogeneity. FedProx is a generalization and re-parameterization of FedAvg that guarantees convergence of the framework, exhibiting more stable and accurate convergence behavior. pFedMe [42] addresses the challenges posed by statistical diversity among clients by proposing a personalized FL (pFedMe) algorithm, which uses the morrow envelope as the client-side regularized loss function. It helps decouple personalized model optimization from global model learning in the two-layer problem of personalized FL stylization. PerAvg [43] was designed to improve its personalization. At the same time, PerAvg explores the impact of the proximity of the distribution of the user database on the personalization of FL.

### 3. Materials and Methods

Our work can be divided into multi-branch classification and vertical blocking aggregation. The framework is shown in Figure 1.



**Figure 1.** The overall framework of our work. Colors differentiate data flow from each other by the client's local labels.

Due to privacy concerns, our framework builds a mobile communication system where clients do not share training data. To identify anomalous network flow, we designed



a multi-branch classification model to detect anomaly flow in the network and distribute the initial model to each client. This model has 23 branches, according to the global label number in our dataset, and each branch is trained to determine whether the input flow belongs to a specific type of label. So, when local data lack a kind of label (which we mark with different colors), the gradient of the corresponding branch is not referential and will not be broadcast. The global server collects each part of the model's gradients from referential clients and calculates their weight according to the local data amount of the specific label. Aggregation is applied separately using a weighted average, and a global gradient is sent to every client to start the next training epoch.

In order to solve the problem of multiple classifications, multi-branch classification constructs an ensemble model. The model includes feature extraction, ensembled binary classification branches, and an output classifier. The model parameters can be separately updated. Raw data are input into the model, and the feature extractor provides intermediate vectors. Each binary classifier utilizes its vectors separately, and all prediction results of these classifiers are integrated into one label prediction.

Under the application environment of FL, not every client has data including all types of attacks. It is useless to train classifiers to look for attacks that a client has never experienced. These branches in the model should be neither trained nor sent in the global learning procedure. In order to determine which branch should be updated and sent, clients should send the server local labels. Feature extractors and output classifiers should constantly be updated. After stepping the parameters, the client should send the model parts, including the feature extractor, output classifier, and effective binary classifiers with corresponding local data.

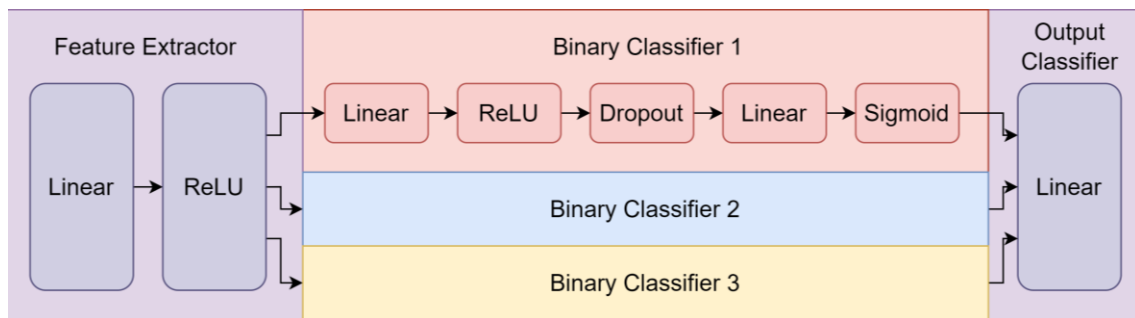
The server receives all model parts from clients and marks their source to prepare for aggregation. It counts each client's influential label as well as the dispersion of the data. It computes the weight metrics as the proportion of the data volume of the current node owning the label to its total practical data volume. This metric describes the effectiveness of each client when updating a specific part of the global model. The weights of the clients that do not have labels are set to 0. Additionally, the server aggregates the uploaded models part by part using the already computed weight metrics. Each part of the model's parameters is computed by averaging the corresponding part of models uploaded by clients.

### 3.1. Multi-Branch Classification

Because anomalous attacks are easy to discover with enough features and proper extraction, we observed that when categorizing data flow distichously, even a naive fully connected neural network performs well. However, detecting a hazardous data flow is typically not enough. A modern security system needs more information about the attack type and possible behavior to react accordingly, thus putting higher requirements on the NIDS to judge the specific types of detected attacks.

The traditional classification method performs poorly on this issue. But when dealing with a divided problem, such as determining if the data flow corresponds to a particular type of attack, the binary classification method performs effectively. Therefore, we suggest a method that ensembles binary classifiers into a single network, which reduces the multi-classification problem into many binary ones.

The KDD99 dataset (<https://www.kaggle.com/datasets/toobajamal/kdd99-dataset> (accessed on 4 July 2023)) includes 22 types of attacks and normal data flow. Therefore, we extend the network by 23 mini-binary classification models. Generally speaking, this can be scaled conveniently by adding the proper amount of mini-networks according to the number of labels in the global dataset. Considering that it is still a multi-classification problem, we apply a feature extractor before the data are processed with the binary classifiers and use a simple 23-dimension classifier to generate the prediction result. Figure 2 depicts the network's organizational structure.



**Figure 2.** The model structure. The model is divided into three main parts: feature extractor, binary classifier, and output classifier.

As shown in Figure 2, our model is divided into three main parts. To reveal feature patterns, the feature extractor is used to adjust the size and structure of pre-processed data. A binary classifier judges whether a flow belongs to a particular label. The output classifier integrates the binary classifier's output and makes a global prediction.

### 3.2. Vertical Blocking Aggregation

Initially, the server connects clients and sets the global environment. To satisfy the needs for aggregation, it creates several client processes after reading the model and method used. The server also initializes optional global parameters like join ratio or time threshold to refuse late updates, which can improve global efficiency.

Clients read local data, summarize label distribution, and mark branches in the model that do not need a gradient that depends on whether the client has the corresponding label. These label parts will not be trained, enabling the client to match the computation power limit more easily. According to the global algorithm, it also initializes its optimizer, loss function, and other necessary components, including the learning rate scheduler.

Then the client trains our model locally and records its local time consumption during the training period. Certain parts of the model are sent to the server, which monitors whether the client is taking too long to complete the training and communication process. The model parts that have not been trained will not be sent, which can reduce communication consumption, as calculated using Equation (1).

$$\text{Communication Volume Rate} = \frac{\text{FEO} + \text{LLNum} * \text{BO} + \text{OCO}}{\text{FEO} + \text{GLNum} * \text{BO} + \text{OCO}}. \quad (1)$$

In Equation (1), FEO is the feature extractor overhead, BO is the branch overhead, and OCO is the output classifier overhead. LLNum is the number of local labels and GLNum is the number of global labels. Since the branch is the most complex part of the model, this rate is approximately LLNum over GLNum.

The server obtains every component of the model parts and calculates the data used to train each component. Then, the server traverses all uploaded weights and computes how much they will affect the global model using the ratio of the data volume owned by the client to all the data volume in this part of the model.

Finally, the server aggregates all uploaded models according to the weight metrics described above. Every round, the server creates a new, empty global model and fills it with the weighted average of certain parts from different clients. The feature extractor and output classifier weights are usually determined by whether the global dataset distributes on each client evenly, and other weights vary significantly when the data are not independently and identically distributed on each node.

The overall process of the algorithm is described using Algorithm 1.

**Algorithm 1.** Vertical Blocking Aggregation.**Input:** Clients  $C_i$  and their local data set  $D_i$ , maximum training round  $T$ .**Output:** the final global model  $w$ .1. **Server executes:**2. Receive label distribution  $L_i$  from clients.3. Initialize global model  $w^0$ .4. **For** communication round  $t < T$ :5.   **For** each client  $C_i$ :6.     Send the effective parts from global model  $w^t$  to  $C_i$  according to  $L_i$ .7.   **For** each client  $C_i$ :8.     Receive the local model  $w_i^{t+1}$  from  $C_i$ .9.   Aggregate the local models to obtain  $w^{t+1}$ .10.   **return** the final global model  $w = w^T$ .11.   **Client executes:**12.   Send label distribution  $L_i$  to server.13.   **For** communication round  $t < T$ :14.     Initialize the effective parts of the local model with  $w^t$  to obtain  $w_i^t$ .15.     Fix the ineffective parts in  $w_i^t$ .16.     Train  $w_i^t$  on  $D_i$  to obtain  $w_i^{t+1}$ .17.     Send  $w_i^{t+1}$  to the server.**4. Results and Discussion**

The Results and Discussion section is divided into three main parts. First, we explain the environment and settings we used; then, we will compare our method with others regarding accuracy and speed. Finally, we describe the other experiments that were carried out and analyze them in light of the results.

**4.1. Settings**

The dataset we used is KDDCup99. This dataset is a feature extraction version of the DARPA dataset, containing feature information from multiple aspects such as connection protocol, content distribution, and traffic statistics. It is a stream-based dataset enriched with additional information from packet-based data or host-based log files, so its data are adequate and representative. These data are available in full dataset volume and 10% data volume. The data have 41 features with five distinct attack classes: Normal, DoS, Probe, R2L, and U2R.

Our experiment is conducted on a seven-device cluster, where each device is a GeForce RTX 2080 Ti GPU. The cluster is built within a local testbed to emulate a distributed cluster from cloud platforms. The implementation of our algorithm is based on the PyTorch distributed library.

For accuracy, we use Equation (2) for the accuracy calculation.

$$\text{TPR} = \frac{\text{TP} + \text{TN}}{\text{TP} + \text{TN} + \text{FP} + \text{FN}} \quad (2)$$

In Equation (2), TP stands for the positive sample that the model predicts will be in the positive class, TN stands for the negative sample that the model predicts will be in the negative class, FP stands for the negative sample predicted by the model to be in the positive class, and FN stands for the positive samples predicted by the model to be in the negative class.

Because aggregation on the server may cause performance decline, we do not use the aggregated model to evaluate the system performance. We evaluate models on each client locally, and the clients broadcast their accuracy. Instead, the global server chooses the best model as the global evaluation model and reports its performance.

The comparative algorithms we use include FedAvg [29], APFL [30], APPLE [31], Ditto [32], FedBN [33], FedFomo [34], FedMTL [35], FedProx [36], pFedMe [37], PerAvg [38],



all of which are based on FL and attempt to improve upon it. In the following section, we present the research and improvement directions for each of these nine algorithms.

Research on FL has focused on three main areas: statistical heterogeneity, system constraints, and trustworthiness. FedAvg is the most classic algorithm for FL which eliminates the need to aggregate all data on a single device using a multi-round learning and communication approach, thus overcoming the privacy and communication challenges of machine learning tasks and allowing machine learning models to learn data stored on individual users (clients) in a decentralized manner. By studying a combination of global and local patterns, APFL also explores personalization. APPLE also investigates aspects of personalization, with a policy that adaptively learns how much each client can benefit from the model of other clients. With applied multi-tasking learning, Ditto eliminates the conflict between fairness and robustness. FedBN addresses the case of feature shift in FL data heterogeneity by adding a batch normalization layer (BN) to the local model. FedFomo enables the server to save each client's model, and clients can successfully download models from the server for use in local target tasks. It also proposes the use of differential privacy to encrypt the data. FedMTL considers the problems of high communication costs, laggards, and fault tolerance of distributed multi-task learning applied to FL. FedProx considers data heterogeneity and system heterogeneity optimizes the objective function and makes the algorithm converge better. In order to improve its customized model, pFedMe provides Moreau envelope optimization based on client-side loss functions. PerAvg is an initial shared model that current or new users can easily adapt to their local dataset by performing one or a few steps of gradient descent concerning their data.

In terms of datasets, given that different algorithms behave differently in two cases of data distribution (independent identically distributed, IID, and non-independent identically distributed, non-IID), we studied both cases as datasets based on the KDD99 dataset. Traditional machine learning should theoretically distribute data on the same machine. They are assumed to be sampled independently from the same distribution, i.e., the data are independently identically distributed. Thus, data are typically IID for the majority of training operations. However, because devices are attributed to a particular user, company, or scenario, their data distribution is often very different, i.e., non-independent identically distributed data (non-IID). Firstly, the data are non-identically distributed because the data distribution varies widely; secondly, the data distribution of these devices is often correlated again, i.e., non-independent, due to factors such as user groups and geographical associations.

We generate different distributions across clients by allocating classes to each client and splitting raw data among each client evenly. All classes are allocated evenly to all clients in the IID situation. In the non-IID situation, we limit the class number on most clients to a fixed amount and allocate classes to each client until it reaches the limit. In order to determine the quality of data for each class after creating the class distribution, we divide the total amount of data for each class after creating the class distribution, we divide the total amount of data among all clients into portions equal to the number of classes on each client, and finally, we split the original data set according to the data division scheme and sent them to every client.

Specifically, we count the amount of all tensors tracked by Python, including parameters, gradients, and data copy. This metric indicates the resource consumption for maintaining and communicating the model. The total tensor of this algorithm will be less if it saves more time and uses less communication traffic. We also calculate the communication volume of each algorithm based on the communication overhead of FedAvg to prove that FedVB can reduce the number of parameters that need to be passed. An aggregation algorithm's communication volume includes gradients and parameters that must be sent. In some algorithms, not every part of the model parameters needs to be broadcast, while other algorithms pass models more than once in an epoch. A better-designed model structure and corresponding aggregation policy can reduce this overhead remarkably.

In the following sections, we compare FedVB with different algorithms in the ‘baseline’ section under IID cases and conduct control experiments by changing the model and dataset distribution in the ‘other experiments’ section. We use the metrics shown above to judge the algorithms’ complexity, accuracy, and communication overhead.

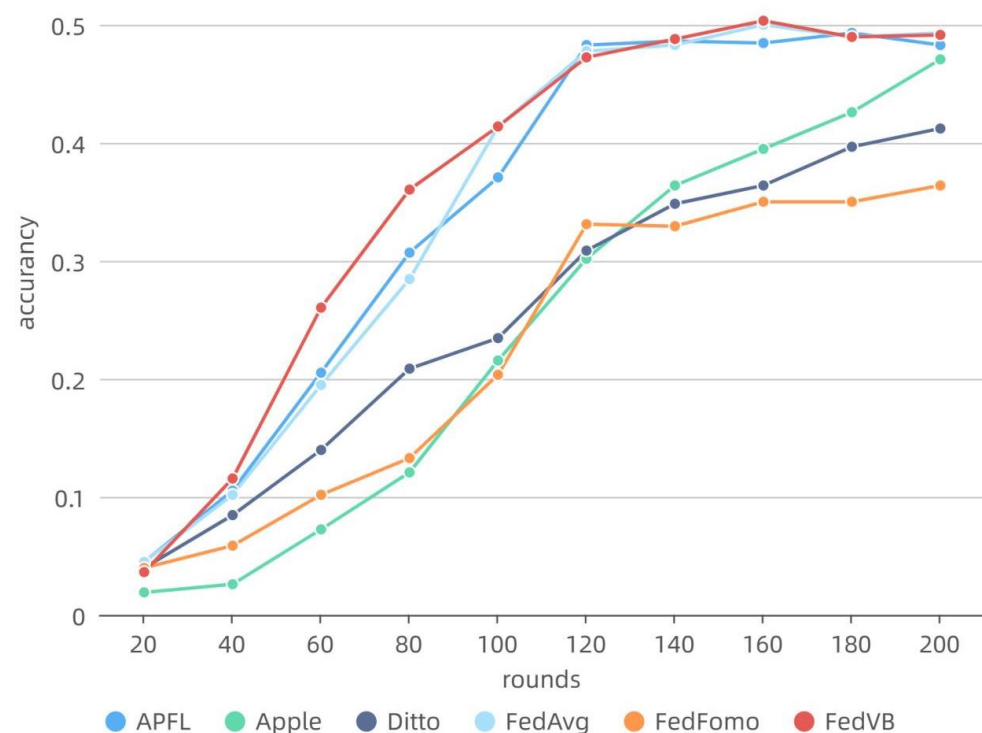
#### 4.2. Baseline

In a specific CFEC framework, the worldwide distribution of data is essentially the same even though a client may face different attacks. More generally, we compared our FedVB algorithm with other aggregation algorithms under IID circumstances.

We use our multi-branch model as the global evaluation model. Our FedVB algorithm performs among the best in accuracy, communication consumption, and total tensor, which indicates that FedVB combines all the advantages of other algorithms. The results are shown in Table 1 and Figure 3.

**Table 1.** Comparison of different algorithms in terms of total tensors, accuracy, and communication volume.

| Algorithm    | Total Tensors    | Accuracy      | Communication Volume |
|--------------|------------------|---------------|----------------------|
| <b>FedVB</b> | <b>2,900,832</b> | <b>0.5034</b> | 1                    |
| APFL         | 5,537,952        | 0.5000        | 1                    |
| APPLE        | 5,537,952        | 0.4707        | 1                    |
| Ditto        | 5,537,952        | 0.4259        | 1                    |
| FedBN        | 2,900,832        | 0.5017        | 1                    |
| FedFomo      | 6,856,712        | 0.3655        | 10                   |
| FedMTL       | 2,946,412        | 0.5017        | 1                    |
| FedAvg       | 2,900,832        | 0.5000        | 1                    |
| FedProx      | 2,923,572        | 0.5017        | 1                    |
| pFedMe       | 2,925,846        | 0.0379        | 1                    |
| LOCAL        | 2,900,546        | 0.8621        | 0                    |
| PerAvg       | 2,900,832        | 0.7483        | 1                    |



**Figure 3.** Comparison of the convergence speed of different algorithms under the multi-branch model. In this case, our algorithm FedVB is in red, and the other colors are the existing FL algorithms.

FedVB demonstrated excellence in terms of accuracy, communication volume, and total tensor. This shows that FedVB is among the best under IID circumstances, especially when computational resources and memory are tightly restricted.

During training, FedVB also showed a faster converge speed than most algorithms in all periods, contributing to better performance when time and compute resources are tightly restricted.

#### 4.3. Other Experiments

To prove that our multi-branch model designed for the FedVB algorithm can be effective under most circumstances, we compared our multi-branch model with a simpler classification model.

We changed the hidden layer size of a straightforward multi-classification network so that the total tensors of the two models remain constant because adding parameters typically improves prediction accuracy. In particular, we referred to FedAvg. In order to ensure a fair comparison, we adjusted the number of parameters in the first layer so that its total tensors were almost the same in both the simple classifier and multi-branch network cases.

The results are shown in Table 2, from which we can observe that our multi-branch model outperforms the simple classifier model in all algorithms and consumes less memory and total tensors. This is because conventional network attacks involve multiple main types of attacks, and the recognition of each type of attack typically relies on different features. Compared with simple networks, the parameters in each branch of the multi-branch network can learn the pattern information of the attack type identified by that branch without being affected by other attack types, thus having higher accuracy.

**Table 2.** Accuracy comparison of different algorithms under two models: the simple classifier and multi-branch network.

| Algorithm | Simple Classifier | Multi-Branch Network |
|-----------|-------------------|----------------------|
| FedAvg    | 0.3983            | 0.5000               |
| APFL      | 0.4276            | 0.5000               |
| Apple     | 0.3983            | 0.4707               |
| Ditto     | 0.3137            | 0.4259               |
| FedBN     | 0.3983            | 0.5017               |
| FedMTL    | 0.3224            | 0.5017               |
| FedProx   | 0.4000            | 0.5017               |
| pFedMe    | 0.0259            | 0.0379               |
| FedFomo   | 0.8897            | 0.8500               |
| LOCAL     | 0.8638            | 0.8621               |
| PerAvg    | 0.6000            | 0.7483               |

Additionally, in some cases, certain attack types rely heavily on specific clients, leading to a non-IID problem. However, traditional FL algorithms usually encounter problems since it is challenging to generalize local learning gradients.

We also compared our algorithm with traditional ones under non-IID cases, and the results indicate that FedVB reduces communication consumption considerably. Our local model is compressed from the perspective of computation and communication since branches without local labels are configured to refuse to be updated and sent. The significant decline in the communication volume of FedVB can strongly support this, which is shown in Table 3.

We generated non-IID data and compared our model and simple classifiers in general algorithms. The comparison indicates that our model's prediction is significantly more accurate, as shown in Table 4.

**Table 3.** Comparison of total tensor data for different algorithms using the non-IID dataset.

| Algorithm    | Total Tensors    | Communication Volume |
|--------------|------------------|----------------------|
| <b>FedVB</b> | <b>1,864,176</b> | 0.2138               |
| APFL         | 5,537,406        | 1                    |
| APPLE        | 5,537,952        | 1                    |
| Ditto        | 5,537,952        | 1                    |
| FedBN        | 2,900,832        | 1                    |
| FedFomo      | 6,856,712        | 10                   |
| FedMTL       | 2,946,412        | 1                    |
| FedAvg       | 2,900,546        | 1                    |
| FedProx      | 2,923,572        | 1                    |
| pFedMe       | 2,925,846        | 1                    |

**Table 4.** Accuracy comparison of different algorithms under two models: the multi-branch network and simple classifier.

| Algorithm | Simple Classifier | Multi-Branch Network |
|-----------|-------------------|----------------------|
| FedAvg    | 0.4190            | 0.5224               |
| APFL      | 0.5155            | 0.5155               |
| Apple     | 0.2810            | 0.2448               |
| Ditto     | 0.1724            | 0.1741               |
| FedBN     | 0.5086            | 0.4086               |
| FedFomo   | 0.1724            | 0.1690               |
| FedMTL    | 0.1672            | 0.1810               |
| FedProx   | 0.5121            | 0.4052               |
| pFedMe    | 0.0862            | 0.0448               |
| FedFomo   | 0.1724            | 0.1690               |
| LOCAL     | 0.1638            | 0.1741               |
| PerAvg    | 0.4931            | 0.3879               |

Since clients do not report model branches to classify labels they do not have, the server can generalize the model more efficiently and accurately than a typical basic classifier.

## 5. Conclusions

This paper shows our work on designing a new aggregation algorithm to reduce hardware requirements for computation and communication. We propose a multi-branch classification model for anomaly detection to accomplish this goal, which can provide a detailed attack type for other network security systems. Our experiments demonstrated that the proposed FedVB algorithm performs among the best across all metrics, and our multi-branch network outperforms simple classifiers in all data sets and algorithms. Under non-IID cases, FedVB can significantly reduce communication overhead compared with traditional FL methods. More research can be conducted on improving the accuracy of classifiers in each branch and designing network structures and metrics to determine effective update gradients.

**Author Contributions:** Conceptualization, Y.W. and H.S.; methodology, Y.W., W.Z. and Z.L.; software, W.Z. and Z.L.; validation, Y.W., W.Z. and Z.L.; formal analysis, Y.W. and J.W.; investigation, Y.W. and J.W.; resources, H.S.; data curation, W.Z.; writing—original draft preparation, W.Z. and Z.L.; writing—review and editing, Y.W., J.W., H.S., M.G. and Y.D.; visualization, W.Z. and Z.L.; supervision, H.S.; project administration, J.W. All authors have read and agreed to the published version of the manuscript.

**Funding:** This research received no funding.

**Data Availability Statement:** The data that support the findings of this study are openly available at <https://www.kaggle.com/datasets/toobajamal/kdd99-dataset> (accessed on 4 July 2023).

**Conflicts of Interest:** The authors declare no conflict of interest.

## References

1. Xu, Y.; Wu, Y.; Gao, H.; Song, S.; Yin, Y.; Xiao, X. Collaborative APIs recommendation for artificial intelligence of things with information fusion. *Future Gener. Comput. Syst.* **2021**, *125*, 471–479. [\[CrossRef\]](#)
2. Li, H.; Tang, B.; Lu, H.; Cheema, M.A.; Jensen, C.S. Spatial data quality in the IoT era: Management and exploitation. In Proceedings of the 2022 International Conference on Management of Data, Philadelphia, PA, USA, 12–17 June 2022; pp. 2474–2482.
3. Moustafa, N.; Slay, J. The significant features of the UNSW-NB15 and the KDD99 data sets for network intrusion detection systems. In Proceedings of the 2015 4th International Workshop on Building Analysis Datasets and Gathering Experience Returns for Security, Kyoto, Japan, 5 November 2015; pp. 25–31.
4. Ye, N.; Emran, S.M.; Chen, Q.; Vilbert, S. Multivariate statistical analysis of audit trails for host-based intrusion detection. *IEEE Trans. Comput.* **2002**, *51*, 810–820. [\[CrossRef\]](#)
5. Gao, H.; Xu, Y.; Yin, Y.; Zhang, W.; Li, R.; Wang, X. Context-aware QoS prediction with neural collaborative filtering for Internet-of-Things services. *IEEE Internet Things J.* **2020**, *7*, 4532–4542. [\[CrossRef\]](#)
6. Wang, X.; Li, H.; Chen, K.; Shou, L. FEDBFT: An efficient federated learning framework for BERT further pre-training. In Proceedings of the 2023 International Joint Conference on Artificial Intelligence, Macao, China, 19–25 August 2023.
7. Nguyen, T.D.; Rieger, P.; Miettinen, M.; Sadeghi, A.R. Poisoning attacks on federated learning-based IoT intrusion detection system. In Proceedings of the Workshop on Decentralized IoT Systems and Security, San Diego, CA, USA, 23–26 February 2020; pp. 1–7.
8. Zhang, R.; Chu, X.; Ma, R.; Zhang, M.; Lin, L.; Gao, H.; Guan, H. OSTTD: Offloading of splittable tasks with topological dependence in multi-tier computing networks. *IEEE J. Sel. Areas Commun.* **2023**, *41*, 555–568. [\[CrossRef\]](#)
9. Shi, H.; Wang, H.; Ma, R.; Hua, Y.; Song, T.; Gao, H.; Guan, H. Robust searching-based gradient collaborative management in intelligent transportation system. *ACM Trans. Multimed. Comput. Commun. Appl.* **2022**. [\[CrossRef\]](#)
10. Mohd, R.Z.; Zuhairi, M.F.; Shadil, A.Z.; Dao, H. Anomaly-based NIDS: A review of machine learning methods on malware detection. In Proceedings of the 2016 International Conference on Information and Communication Technology, Kuala Lumpur, Malaysia, 12 October 2016; pp. 266–270.
11. Shone, N.; Ngoc, T.N.; Phai, V.D.; Shi, Q. A deep learning approach to network intrusion detection. *IEEE Trans. Emerg. Top. Comput. Intell.* **2018**, *2*, 41–50. [\[CrossRef\]](#)
12. Gao, H.; Xiao, J.; Yin, Y.; Liu, T.; Shi, J. A mutually supervised graph attention network for few-shot segmentation: The perspective of fully utilizing limited samples. *IEEE Trans. Neural Netw. Learn. Syst.* **2022**, 1–13. [\[CrossRef\]](#)
13. Mirlekar, S.; Kanojia, K.P. A comprehensive study on machine learning algorithms for intrusion detection system. In Proceedings of the 2022 10th International Conference on Emerging Trends in Engineering and Technology-Signal and Information Processing, Nagpur Nagpur, India, 29–30 April 2022; pp. 1–6.
14. Kuk, K.; Milić, P.; Denić, S. Object-oriented software metrics in software code vulnerability analysis. In Proceedings of the 2020 International Conference on INnovations in Intelligent SysTems and Applications, Novi Sad, Serbia, 24–26 August 2020; pp. 1–6.
15. Aslanpour, M.S.; Gill, S.S.; Toosi, A.N. Performance evaluation metrics for cloud, fog and edge computing: A review, taxonomy, benchmarks and standards for future research. *Internet Things* **2020**, *12*, 100273. [\[CrossRef\]](#)
16. Ashouri, M.; Lorig, F.; Davidsson, P.; Spalazzese, R. Edge computing simulators for iot system design: An analysis of qualities and metrics. *Future Internet* **2019**, *11*, 235. [\[CrossRef\]](#)
17. Gao, H.; Huang, W.; Liu, T.; Yin, Y.; Li, Y. PPO2: Location privacy-oriented task offloading to edge computing using reinforcement learning for intelligent autonomous transport systems. *IEEE Trans. Intell. Transp. Syst.* **2023**, *24*, 7599–7612. [\[CrossRef\]](#)
18. Rouf, Y.; Shtern, M.; Fokaefs, M.; Litoiu, M. A hierarchical architecture for distributed security control of large scale systems. In Proceedings of the 2017 IEEE/ACM 39th International Conference on Software Engineering Companion, Buenos Aires, Argentina, 20–28 May 2017; pp. 118–120.
19. Ishibashi, R.; Goto, H.; Han, C.; Ban, T.; Takahashi, T.; Takeuchi, J. Which packet did they catch? Associating NIDS alerts with their communication sessions. In Proceedings of the 2021 16th Asia Joint Conference on Information Security, Seoul, Republic of Korea, 19–20 August 2021; pp. 9–16.
20. Rathore, M.M.; Ahmad, A.; Paul, A. Real time intrusion detection system for ultra-high-speed big data environments. *J. Supercomput.* **2016**, *72*, 3489–3510. [\[CrossRef\]](#)
21. Lin, C.H.; Hsieh, C.H. A novel hierarchical parallelism for accelerating NIDS using GPUs. In Proceedings of the 2018 IEEE International Conference on Applied System Invention, Chiba, Japan, 13–17 April 2018; pp. 578–581.
22. Sato, H.; Kobayashi, R. A machine learning-based NIDS that collects training data from within the organization and updates the discriminator periodically and automatically. In Proceedings of the 2021 Ninth International Symposium on Computing and Networking Workshops, Matsue, Japan, 23–26 November 2021; pp. 420–423.
23. Amutha, S.; Kavitha, R.; Srinivasan, R.; Kavitha, M. Secure network intrusion detection system using NID-RNN based deep learning. In Proceedings of the 2022 International Conference on Advances in Computing, Communication and Applied Informatics, Chennai, Tamilnadu, India, 28–29 January 2022; pp. 1–5.
24. Poltavtseva, M.A.; Zegzhda, D.P.; Pavlenko, E.Y. High-performance NIDS architecture for enterprise networking. In Proceedings of the 2019 IEEE International Black Sea Conference on Communications and Networking, Sochi, Russia, 3–6 June 2019; pp. 1–3.



25. Amoli, P.V.; Hämäläinen, T. A real time unsupervised NIDS for detecting unknown and encrypted network attacks in high speed network. In Proceedings of the 2013 IEEE International Workshop on Measurements & Networking, Naples, Italy, 7–8 October 2013; pp. 149–154.
26. Al Haddad, Z.; Hanoune, M.; Mamouni, A. A collaborative framework for intrusion detection (C-NIDS) in cloud computing. In Proceedings of the 2016 2nd International Conference on Cloud Computing Technologies and Applications, Marrakech, Morocco, 24–26 May 2016; pp. 261–265.
27. Liu, L.; Engelen, G.; Lynar, T.; Essam, D.; Joosen, W. Error prevalence in NIDS datasets: A case study on CIC-IDS-2017 and CSE-CIC-IDS-2018. In Proceedings of the 2022 IEEE Conference on Communications and Network Security, Austin, TX, USA, 3–5 October 2022; pp. 254–262.
28. Shenfield, A.; Day, D.; Ayesh, A. Intelligent intrusion detection systems using artificial neural networks. *ICT Express* **2018**, *4*, 95–99. [[CrossRef](#)]
29. Bharati, M.P.; Tamane, S. NIDS-network intrusion detection system based on deep and machine learning frameworks with CICIDS2018 using cloud computing. In Proceedings of the 2020 International Conference on Smart Innovations in Design, Environment, Management, Planning and Computing, Aurangabad, India, 30–31 October 2020; pp. 27–30.
30. Liu, Y.; Li, Z.; Zhou, C.; Jiang, Y.; Sun, J.; Wang, M.; He, X. Generative adversarial active learning for unsupervised outlier detection. *IEEE Trans. Knowl. Data Eng.* **2020**, *32*, 1517–1528. [[CrossRef](#)]
31. Gao, H.; Dai, B.; Miao, H.; Yang, X.; Barroso, R.J.D.; Walayat, H. A novel GAPG approach to automatic property generation for formal verification: The GAN perspective. *ACM Trans. Multimed. Comput. Commun. Appl.* **2023**, *19*, 1–22. [[CrossRef](#)]
32. Guo, H.; Wang, H.; Song, T.; Hua, Y.; Lv, Z.; Jin, X.; Xue, Z.; Ma, R.; Guan, H. Siren: Byzantine-robust federated learning via proactive alarming. In Proceedings of the ACM Symposium on Cloud Computing, Seattle, WA, USA, 1–4 November 2021; pp. 47–60.
33. Zhang, J.; Hua, Y.; Wang, H.; Song, T.; Xue, Z.; Ma, R.; Guan, H. FedALA: Adaptive local aggregation for personalized federated learning. In Proceedings of the 37th AAAI Conference on Artificial Intelligence, Washington, DC, USA, 7–14 February 2023; pp. 11237–11244.
34. McMahan, B.; Moore, E.; Ramage, D.; Hampson, S.; y Arcas, B.A. Communication-efficient learning of deep networks from decentralized data. In Proceedings of the 20th International Conference on Artificial Intelligence and Statistics, Fort Lauderdale, FL, USA, 20–22 April 2017; pp. 1273–1282.
35. Deng, Y.; Kamani, M.M.; Mahdavi, M. Adaptive personalized federated learning. *arXiv* **2020**, arXiv:2003.13461.
36. Luo, J.; Wu, S. Adapt to adaptation: Learning personalization for cross-silo federated learning. In Proceedings of the 31st International Joint Conference on Artificial Intelligence, Vienna, Austria, 23–29 July 2022; pp. 2166–2173.
37. Li, T.; Hu, S.; Beirami, A.; Smith, V. Ditto: Fair and robust federated learning through personalization. In Proceedings of the 38th International Conference on Machine Learning, Virtual Event, 18–24 July 2021; pp. 6357–6368.
38. Li, X.; Jiang, M.; Zhang, X.; Kamp, M.; Dou, Q. FedBN: Federated learning on Non-IID features via local batch normalization. In Proceedings of the 9th International Conference on Learning Representations, Virtual Event, 3–7 May 2021.
39. Zhang, M.; Sapra, K.; Fidler, S.; Yeung, S.; Alvarez, J.M. Personalized federated learning with first order model optimization. In Proceedings of the 9th International Conference on Learning Representations, Virtual Event, 3–7 May 2021.
40. Smith, V.; Chiang, C.K.; Sanjabi, M.; Talwalkar, A.S. Federated multi-task learning. In Proceedings of the Advances in Neural Information Processing Systems 30: Annual Conference on Neural Information Processing Systems, Long Beach, CA, USA, 4–9 December 2017; pp. 4424–4434.
41. Li, T.; Sahu, A.K.; Zaheer, M.; Sanjabi, M.; Talwalkar, A.; Smith, V. Federated optimization in heterogeneous networks. *Proc. Mach. Learn. Syst.* **2020**, *2*, 429–450.
42. T Dinh, C.; Tran, N.; Nguyen, J. Personalized federated learning with moreau envelopes. In Proceedings of the Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems, Virtual Event, 6–12 December 2020.
43. Fallah, A.; Mokhtari, A.; Ozdaglar, A. Personalized federated learning with theoretical guarantees: A model-agnostic meta-learning approach. In Proceedings of the Advances in Neural Information Processing Systems 33: Annual Conference on Neural Information Processing Systems, Virtual Event, 6–12 December 2020.

**Disclaimer/Publisher’s Note:** The statements, opinions and data contained in all publications are solely those of the individual author(s) and contributor(s) and not of MDPI and/or the editor(s). MDPI and/or the editor(s) disclaim responsibility for any injury to people or property resulting from any ideas, methods, instructions or products referred to in the content.